# RST_app Documentation

Arthur Barros

Invalid Date

## Table of contents



### Summary


The `RST_app.R` Shiny application estimates juvenile salmonid out-migration passage (abundance) using rotary screw trap data submitted by the user. This application aims to replicate passage calculation and imputation methods utilized in the CAMP Rotary Screw Trap (CAMPR) platform but in a standalone, user friendly , documented, and replicable online platform. The CAMPR platform was created by Trent McDonald to support CDFW salmonid monitoring

efforts in the Central Valley and allows direct connection between the modeling software built in R and the monitoring Access database. While CAMPR provides a usable system to simplify complex model fitting and passage estimation, it is currently a “black-box” back end with undetailed documentation and is no longer maintained or updated.

The `RST_app.R` is a standalone package and RShiny application that performs model fitting and passage estimation in a web-based platform that requires no Access database or R experience to operate. While it does not require database connectivity, users do need to ensure their data is pre-formatted for use with the application. This document also aims to be a detailed overview of use, methodology, and the relevant scripts and functions that are used by the application. This application is also owned and maintained by CDFW staff, which allows for future maintenance and updates as needed.

Figure 1 shows an overview of how data and user inputs are utilized by the different functions in the `RST_app` to produce estimates of passage, efficiency, catch, and related summary plots. Users need to upload .csv files of `catch_data`, `visit_data`, `release_data`, and `recapture_data` to the application. Then users can either set or use default input values to determine how passage data is summarized, whether or not to impute all catch or efficiency values, and the start and end dates for the survey period. These inputs are passed to the `est_passage.R` function, which then utilizes several downstream functions to produce estimates of passage and related uncertainty. The data inputs and relevant functions are covered in more detail in the following sections of this methods documentation.

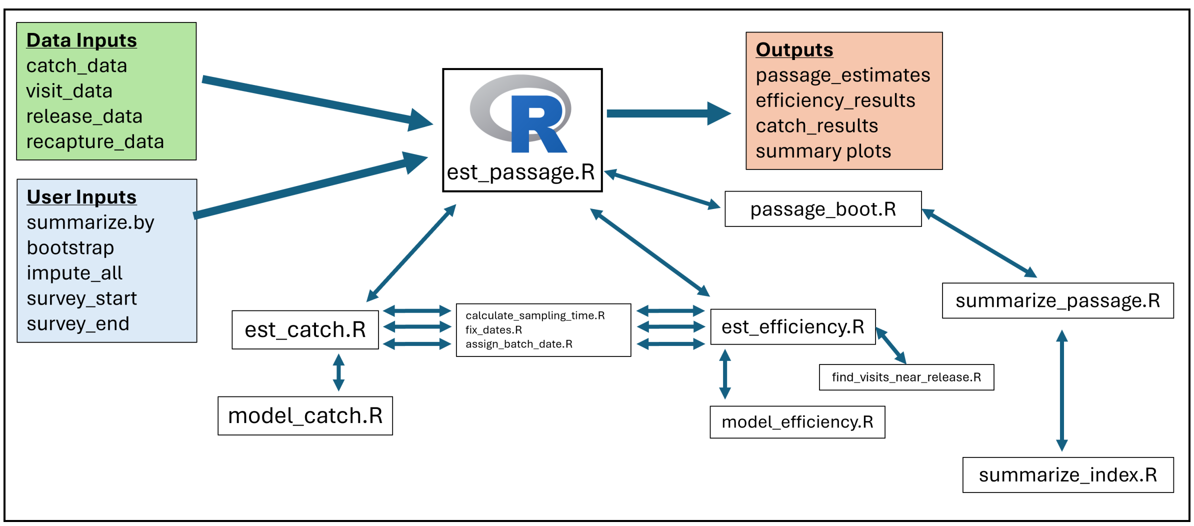


Figure 1: Overview of data inputs, user inputs, functions, and application outputs for RST_app.

Code, documentation, and example data for the `RST_app` can be found at: https://github.com/arthurbarros-CDFW/RST_app.

## RST_app User Guide

The `RST_app` is hosted and can be accessed by users at https://arthurbarros-cdfw.shinyapps.io/RST_app/. The application is currently hosted on a free account using the shinyapps.io platform which limits usage hours per month, but I aim to have it hosted on a paid account to alleviate that in the future. To use the `RST_app` users need to have four datasets in a specific .csv file format. Structuring the application to allow for data inputs in different formats would be inefficient and make it unwieldy. It is instead simpler to construct the application to intake data in a specific format to ensure compatibility, so the onus is on the user to ensure proper their data is in the proper structure.

### *Data formatting*

The user is required to input data from four .csv tables:

- catch - records of fish caught during each trap visit.
- visit - records of all trap operation and sampling events.
- release - records of fish marked and released for efficiency trials.
- recapture - records of marked fish recaptured for efficiency trials.

When uploading the four .csv files, the `app.R` script looks for the keywords "catch", "visit", "release", and "recapture" in the file names, and assigns the files to relevant data set variable names, so if users ensure those keywords are in the appropriate file name, the application will assign the data properly for use in relevant functions.

For examples of how to structure the input data, see https://github.com/arthurbarros-CDFW/RST_app/tree/main/data/test_data. Below I detail which fields are required within the tables for the application to function properly. Tables can include additional data, and additional fields may be useful in the future as the applications functionality is updated.

**`catch.csv`** needs to be set in the following structure:

| Field | Type | Description | Required (Y/N) |
|---|---|---|---|
| site_name | text | Name of river/study site | Y |
| sub-site_name | text | Trap location identifier used when multiple traps at a site. Still use if only one trap at site. | Y |
| visit_time | datetime (ISO 8601) | Date and time when trap was visited and catch recorded. YYYY-MM-DDThh:mm:ss | Y |
| common_name | text | Species common name, currently only used for "Chinook salmon". | Y |

| Field | Type | Description | Required (Y/N) |
|---|---|---|---|
| at_capture_run | text | Run timing, currently only used for "Fall". | N |
| n | integer | Number of fish captured by trap and recorded during visit. | Y |

**visit.csv** needs to be structured in the following format:

| Field | Type | Description | Required (Y/N) |
|---|---|---|---|
| site_name | text | Name of river/study site | Y |
| subsite_name | text | Trap location identifier used when multiple traps at a site. Still use if only one trap at site. | Y |
| visit_time | datetime (ISO 8601) | Date and time when trap was visited and catch recorded. YYYY-MM-DDThh:mm:ss | Y |
| trap_visit_ID | text/integer | Unique identifier for each visit | Y |
| visit_type | text | See valid values below | Y |
| fish_processed | text | Processing status | Y |
| include_catch | text | "Yes" or "No" | Y |

Valid `visit_type` values are:

- `"Start trap & begin trapping"`
- `"Continue trapping"`
- `"End trapping"`
- `"Unplanned Restart"`

Valid `fish_processed` values are:

- `"N/A; not a sampling visit"`
- `"Processed fish"`
- `"No fish were caught"`
- `"No catch data; fish released"`

`release.csv` needs to be structured in the following format:

| Field | Type | Description | Required (Y/N) |
|---|---|---|---|
| site_name | text | Name of river/study site | Y |
| subsite_name | text | Trap location identifier used when multiple traps at a site. Still use if only one trap at site. | Y |
| release_time | datetime (ISO 8601) | Date and time when release occurred YYYY-MM-DDThh:mm:ss | Y |
| release_ID | text/integer | Unique identifier for release event | Y |
| include_analysis | text | "Yes" or "No" used to indicate if data should be used for efficiency estimation | Y |
| n_released | integer | Number of fish released | Y |

`recapture.csv` needs to be structured in the following format:

| Field | Type | Description | Required |
|---|---|---|---|
| site_name | text | Name of river/study site | Y |
| subsite_name | text | Trap location identifier used when multiple traps at a site. Still use if only one trap at site. | Y |
| visit_time | datetime (ISO 8601) | Date and time when recapture occurred YYYY-MM-DDThh:mm:ss | Y |
| trap_visit_ID | text/integer | Links recapture data to visit record | Y |
| release_ID | text/integer | Links recapture data to release record | Y |
| n | integer | Number of recaptured fish | Y |

It is important the user ensures they have the four required data tables available and in the above format in order to properly use the `RST_app`.

### *Inputs*

fill

### *Outputs*

fill

## Functions Overview

fill

### *global_vars.R*

fill

### *fix_dates.R*

### *assign_batch_date.R*

### *calculate_sampling_time.R*

### *est_catch.R*

### *model_catch.R*

### *est_efficiency.R*

### *model_efficiency.R*

### *over_dispersion.R*

### *find_visits_near_release.R*

### *summarize_index.R*

### *summarize_passage.R*

### *passage_boot.R*

### *est_passage.R*

### *app.R*